

# Flujo de trabajo reproducible en R para ecoacústica

Guía de prácticas: de la tarjeta SD a la tabla de análisis

Guillermo Fandos · Universidad Complutense de Madrid · gfandos@ucm.es

Reunión REIE · 26 de junio de 2026

## Qué vamos a hacer hoy

Esta guía te lleva, paso a paso, por una práctica de procesamiento de datos en ecoacústica con R. Está pensada para gente **sin experiencia previa en R**, así que no damos nada por supuesto.

El objetivo es transformar unas grabaciones de audio (tal y como salen de la tarjeta SD de un grabador de campo) en una **tabla de datos limpia y lista para el análisis**, usando un flujo de trabajo *reproducible*: cualquier persona que siga los mismos pasos obtiene exactamente el mismo resultado.

Al final de la sesión serás capaz de:

- Descargar los materiales e instalar el entorno de trabajo.
- Ejecutar un flujo completo con una sola instrucción y entender lo que hace.
- Detectar tres tipos de errores muy frecuentes en proyectos de ecoacústica: errores en los metadatos, en el control de calidad y en la unión de tablas.
- Entender para qué sirve **targets** y por qué vale la pena usarlo.

Sigue los apartados en orden. Las instrucciones de código que aparecen en **tipografía de máquina**, como `targets::tar_make()`, se escriben tal cual en R (o se copian y pegan).

---

## Descarga de los materiales

Los materiales de la práctica están en GitHub:

<https://github.com/guifandos/acoustic-workflow-REIE>

**Si no has usado GitHub antes, descarga el ZIP:**

1. Abre esa dirección en el navegador.
2. Pulsa el botón verde «**Code**» (parte superior derecha de la página).
3. Elige «**Download ZIP**».
4. Descomprime el archivo. Se crea una carpeta llamada **acoustic-workflow-REIE-main**.
5. Mueve esa carpeta a un sitio estable (por ejemplo, tus Documentos) y, si quieres, renómbrala a **acoustic-workflow-REIE**. Esta carpeta es el **directorio del proyecto**; necesitarás su ubicación más adelante.

Si tienes **Git** instalado, puedes clonarlo directamente desde una terminal:

```
git clone https://github.com/guifandos/acoustic-workflow-REIE.git
```

---

## Instalación del entorno

Si ya tienes R (versión  $\geq 4.2$ ) y RStudio instalados, ve directamente al apartado **Instalación de paquetes**.

### Paso 1: Instala R

1. Ve a <https://cran.r-project.org> y selecciona tu sistema operativo.
2. En **Windows**, pulsa «base» y descarga el instalador. En **macOS**, descarga el **.pkg** correspondiente a tu equipo (comprueba si tienes chip Apple Silicon o Intel).
3. Ejecuta el instalador con las opciones por defecto.

### Paso 2: Instala RStudio

1. Ve a <https://posit.co/download/rstudio-desktop/>.
2. Descarga **RStudio Desktop** (versión gratuita) para tu sistema operativo.
3. Instala con las opciones por defecto.

### Paso 3: Primer arranque

Abre RStudio (no hace falta abrir R por separado; RStudio lo usa internamente). Verás varios paneles. El que más vas a usar hoy es la **Consola** (*Console*), donde escribes las instrucciones de R y ves los resultados.

### Paso 4: Instala los paquetes necesarios

Los paquetes son extensiones de R con funciones adicionales. Copia lo siguiente en la Consola y pulsa Intro. Solo tienes que hacerlo una vez; puede tardar unos minutos:

```
install.packages(c("targets", "tarchetypes", "tuneR", "seewave",  
                  "data.table", "lubridate", "dplyr", "stringr"))
```

---

| Paquete                   | Para qué sirve en esta práctica                       |
|---------------------------|-------------------------------------------------------|
| targets, tarchetypes      | Orquesta el flujo y recalcula solo lo que ha cambiado |
| tuneR, seewave            | Lee archivos WAV y calcula índices acústicos          |
| dplyr, lubridate, stringr | Manejo de tablas, fechas y texto                      |

---

## Puesta en marcha

### Sitúa el directorio de trabajo

R necesita saber en qué carpeta están los materiales. Este es el error más frecuente, así que préstale atención.

En RStudio, ve a *Session* → *Set Working Directory* → *Choose Directory...* y selecciona la carpeta del proyecto que descargaste (la que contiene el archivo `_targets.R`).

Para comprobar que todo está en orden, escribe esto en la Consola:

```
getwd()      # debe terminar en el nombre de la carpeta del proyecto
list.files()  # debe mostrar _targets.R, R/, data_raw/, slides/, etc.
```

Si `list.files()` muestra esos nombres, estás en el sitio correcto.

### Abre el script de la práctica

En el panel *Files* de RStudio (normalmente abajo a la derecha), busca y haz clic en el archivo `practica.R`. Se abrirá en el editor. Este script tiene los cuatro ejercicios: irás ejecutando sus bloques línea a línea con **Ctrl+Enter**, o seleccionas un bloque entero y lo corres de una vez.

---

## Ejercicios

Hay cuatro ejercicios. Después de cada uno, deshaz la modificación que hayas introducido y vuelve a ejecutar `targets::tar_make()`, para que el siguiente ejercicio parta del estado original.

### Ejercicio 1: Ejecutar el flujo y examinar la tabla

Ejecuta el bloque del **Ejercicio 1** en `practica.R`:

```
targets::tar_make()
tabla <- read.csv("output/tabla_analisis.csv")
View(tabla)
nrow(tabla)
names(tabla)
```

**Lo que verás.** En la consola aparece el resumen del control de calidad:

Archivos: 6 | Duración anómala: 1 | PASAN QC: 5 / 6

La tabla tiene **5 filas, no 6**. La grabación `20260501_062000_TRUNCADA.WAV` del grabador AM02 (3 segundos en lugar de los 10 esperados) ha sido descartada automáticamente. Cada fila integra tres tipos de información:

- **Señal de audio:** `rms`, `spectral_entropy`
- **Metadatos de la grabación:** `datetime`, `recorder_id`, `duration_s`
- **Diseño de muestreo:** `site`, `habitat`, `lat`, `lon`

Una sola instrucción, desde los WAV crudos hasta una tabla con todo integrado.

**Para pensar:** ¿cómo sabrías, sin este flujo, que falta una grabación?

---

## Ejercicio 2: Una zona horaria incorrecta

El flujo extrae la fecha de cada grabación de dos fuentes: el **nombre del archivo** (20260501\_060000.WAV) y la **cabecera interna del WAV**. Luego las compara. Si no coinciden, avisa. En este ejercicio lo vamos a romper a propósito.

**Paso 1.** Abre `_targets.R` desde el panel *Files* de RStudio (haz clic en el archivo en la lista). Localiza la línea:

```
recorder_tz <- "UTC"
```

Cámbiala a:

```
recorder_tz <- "America/New_York"
```

**Paso 2.** Guarda el archivo con **Ctrl+S**. Si la barra de título de RStudio muestra un punto negro junto al nombre del archivo, significa que hay cambios sin guardar; asegúrate de guardar antes de ejecutar.

Vuelve a  `practica.R` y ejecuta el bloque del Ejercicio 2:

```
targets::tar_make()
```

**Lo que verás:**

DISCREPANCIA nombre vs cabecera en 20260501\_060000.WAV (revisa zona horaria o renombrado).  
Discrepancia nombre/hdr: 6

Si miras las columnas `dt_name` y `dt_header`, las dos muestran “06:00:00”, y aun así el aviso se activa. ¿Por qué? Las 6 de la mañana en Nueva York y las 6 de la mañana en UTC son momentos distintos, separados por horas. Una zona horaria incorrecta desplaza todas las marcas temporales sin producir ningún error en R. Solo la comparación cruzada entre las dos fuentes lo detecta.

Si no ves ningún aviso, es probable que `targets` haya reutilizado la caché anterior. Ejecuta esto y vuelve a intentarlo:

```
targets::tar_invalidate(meta_raw)
targets::tar_make()
```

*Restauración:* vuelve a `recorder_tz <- "UTC"`, guarda y ejecuta `tar_make()`. Las discrepancias deben volver a cero.

---

## Ejercicio 3: El join que infla filas

Este es uno de los errores más frecuentes y difíciles de detectar cuando integras resultados con el diseño de muestreo: la tabla de emplazamientos tiene **dos filas para el mismo grabador** (por una doble clasificación de hábitat, una fila duplicada accidental, o datos de dos técnicos sin reconciliar), y cada grabación de ese grabador se multiplica en la tabla final **sin ningún mensaje de error de R**.

El flujo incluye un guardarraíl que lo detecta. Aquí lo vas a provocar para ver cómo funciona.

**Paso 1.** Abre (o vuelve a) `_targets.R` y localiza la línea:

```
tar_target(sites_file, "data/sites.csv", format = "file"),
```

Cámbiala a:

```
tar_target(sites_file, "data/sites_duplicate.csv", format = "file"),
```

**Paso 2.** Guarda y ejecuta el bloque del Ejercicio 3 en `practica.R`:

```
targets::tar_make()
```

**Lo que verás.** Dos avisos en la consola:

Detected an unexpected many-to-many relationship between `x` and `y`.

El join cambió el nº de filas (5 -> 7). Revisa claves duplicadas en 'sites' o 'meta'.

La tabla tiene **7 filas en lugar de 5**. Las grabaciones del grabador AM02 aparecen duplicadas: una fila con hábitat **matorral** y otra con **pastizal**. Los índices acústicos y las fechas son correctos en las dos filas; el error solo se ve en el número de filas.

Para encontrar qué fila está duplicada:

```
sites_dup <- read.csv("data/sites_duplicate.csv")
sites_dup[duplicated(sites_dup$recorder_id), ]
```

Si el aviso no aparece, `targets` puede haber reutilizado `final_table` de la caché. Ejecuta esto y repite:

```
targets::tar_invalidate(final_table)
targets::tar_make()
```

*Restauración:* vuelve a "data/sites.csv", guarda y ejecuta `tar_make()`. La tabla vuelve a 5 filas.

---

## Ejercicio 4: Cambiar el umbral de control de calidad

Este ejercicio muestra la capacidad más útil de `targets` en el día a día: si modificas algo, **solo recalcula lo que depende de ese algo**. El resto se reutiliza de la caché sin ejecutarse de nuevo.

**Paso 1.** Abre `R/qc.R` desde el panel *Files* (carpeta `R/` → archivo `qc.R`). Localiza la función `qc_flag()`. La primera línea es:

```
qc_flag <- function(meta, expected_dur_s = NULL, tol_frac = 0.05) {
```

El parámetro `tol_frac = 0.05` significa que se descartan las grabaciones con una duración que se desvíe más del 5% de lo esperado. Cámbialo a 0.75:

```
qc_flag <- function(meta, expected_dur_s = NULL, tol_frac = 0.75) {
```

**Paso 2.** Guarda el archivo (Ctrl+S) y ejecuta el bloque del Ejercicio 4 en `practica.R`:

```
targets::tar_make()
```

**Lo que verás.** En la consola, algunos targets aparecen como `skip` (se reutilizan de la caché) y otros se recalculan. El parseo de los metadatos (la etapa más costosa) no se vuelve a ejecutar. El resultado cambia:

PASAN QC: 6 / 6

La tabla tiene ahora **6 filas**: la grabación truncada ya no se descarta porque el umbral es ahora mucho más permisivo.

Un umbral de 0.75 significa que se aceptan grabaciones con hasta el 75% de duración anómala. ¿Tiene sentido para tu proyecto? Eso es una decisión ecológica, no técnica.

*Restauración*: vuelve a `tol_frac = 0.05`, guarda y ejecuta `tar_make()`. Vuelves a PASAN QC: 5 / 6.

---

## Solución de problemas

| Síntoma                                                               | Qué ha pasado                                                         | Qué hacer                                                                                                                                                             |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Error: there is no package called '...' cannot open file '_targets.R' | Falta instalar ese paquete<br>El directorio de trabajo no es correcto | <code>install.packages("nombre")</code> en la Consola<br><i>Session</i> → <i>Set Working Directory</i> → <i>Choose Directory</i> y selecciona la carpeta del proyecto |
| El Ejercicio 2 no produce avisos                                      | No guardaste el archivo, o <code>targets</code> usó la caché          | Guarda con Ctrl+S y ejecuta <code>targets::tar_invalidate(meta_raw); targets::tar_make()</code>                                                                       |
| El Ejercicio 3 no produce aviso del join                              | <code>final_table</code> en caché                                     | <code>targets::tar_invalidate(final_table); targets::tar_make()</code>                                                                                                |
| Aviso sobre <code>sonicscrewdriver</code>                             | Ese paquete está fuera de CRAN desde junio 2026                       | Normal; el flujo funciona igualmente con la implementación local                                                                                                      |
| Fechas con valor NA                                                   | La zona horaria está mal escrita                                      | Usa nombres IANA válidos; consulta <code>OlsonNames()</code> en R                                                                                                     |
| La tabla tiene más filas de las esperadas                             | Clave duplicada en la tabla de emplazamientos                         | <code>sites[duplicated(sites\$recorder_id), ]</code> para localizar la fila                                                                                           |

---

## Recursos y para saber más

Todo el material, los enlaces y las lecturas recomendadas están en `docs/recursos.md` dentro del repositorio. Lo más relevante para continuar:

- **warbleR**, **Rraven**, **ohun**: análisis de estructura de señales en R (espectrogramas por lotes, medidas acústicas, integración con Raven).
- **NSNSDAcoustics**: BirdNET desde R, con funciones de verificación de detecciones incluidas.
- **scikit-maad** (Python): más de 50 índices acústicos; accesible desde R con **reticulate**.
- **soundecology** (R): índices clásicos como ACI, ADI o NDSI; archivado en CRAN desde 2020, sigue funcionando.

- **targets** y **renv**: orquestación y fijación de versiones para la reproducibilidad.
- 

## La idea central

R no hace todo el análisis. La detección y clasificación de especies la hacen modelos especializados, normalmente en Python. Lo que R hace especialmente bien es **coordinar**: gestionar metadatos, controlar la calidad de los datos, integrar resultados y garantizar que todo es reproducible. El principal obstáculo en estos proyectos no es el modelo sofisticado, sino lo que viene antes (los nombres de archivo, los metadatos, la organización de los datos), que es exactamente lo que hemos trabajado hoy.